

Prompts as Production Code

Task 1.6. Instruction design, versioning, governance, regression testing and Bedrock Flows.

The prompt is the highest-leverage code in the system, and it is usually the only part with **no version control, no tests and no review.**

What actually improves output

- ◆ **Specify the format precisely.** "Three bullets, one sentence each" beats "summarise this".
- ◆ **Structured input.** Delimit sections so instruction, data and example are distinguishable.
- ◆ **Few-shot examples.** Two or three beat a paragraph describing what you want.
- ◆ **Chain of thought.** For multi-step reasoning, not for simple extraction.
- ◆ **Feedback loops.** Accepted outputs become tomorrow's examples.

ON THE EXAM

Format specificity is the cheapest reliable improvement available. Vague instructions produce variable output, and variable output breaks whatever parses it downstream.

System prompt or user message?

SYSTEM BLOCK

Stable, and it stays

- Role, guidelines, constraints, tone
- Survives conversation history trimming
- The stable prefix prompt caching needs
- Changed by deployment, not by a user

USER MESSAGE

Variable, per request

- The actual question and its data
- Retrieved chunks and context
- Anything that differs call to call
- Never put instructions only here

THE TIEBREAKER

Instructions in the user turn get trimmed away first in a long conversation, and they break prompt caching. Both failures appear gradually, which makes them hard to notice.

Getting structure you can rely on

- ◆ **JSON Schema** or a tool definition constrains the shape far harder than asking politely for JSON.
- ◆ **Low temperature**, as module 1 established. Structure and creativity pull in opposite directions.
- ◆ **Stop sequences** end generation cleanly at a delimiter you control.
- ◆ **Check *stopReason***. A value of *max_tokens* means your JSON is truncated and will not parse.
- ◆ **Always validate**. Schema-constrained output is far more reliable, not guaranteed.

ON THE EXAM

Text-to-SQL is the extreme version of this: rather than trusting a generated answer, generate a query, run it deterministically, and let the database produce the number.

A logistics company extracts six fields from delivery exception emails and writes them to DynamoDB. The prompt asks the model to reply in JSON. Temperature is set to 0.7 because a previous team member wanted varied phrasing in a different feature and changed it globally.

Around 4% of responses fail to parse. Some include an explanatory sentence before the JSON, some use different key names, and some are truncated mid-object. The team has been catching failures with a retry loop that re-invokes the model up to three times.

Which combination of changes best addresses the parsing failures?

- | | |
|--|---|
| A Increase the retry limit from three to ten and log the failures for later analysis | B Set temperature near zero, constrain the output with a JSON Schema or tool definition, and check stopReason for max_tokens |
| C Add "Reply only with valid JSON and nothing else" in capital letters at the end of the prompt | D Switch to a larger foundation model with stronger instruction-following |

◆ PAUSE AND DECIDE

Which combination of changes best addresses the parsing failures?

A More retries

Retries a non-deterministic call and hopes. It also multiplies token cost on exactly the requests that were already failing.

B Low temperature, schema constraint, check stopReason

Each change maps to one stated symptom: temperature to key-name variation, schema to the preamble, stopReason to the truncation.

C Emphatic instruction in capitals

Prompt wording nudges behaviour. A schema constrains it. Emphasis does not make a probabilistic output deterministic.

D Larger model

Improves instruction-following somewhat, at higher cost, and leaves temperature at 0.7 and truncation unhandled.

THE SIGNAL

Three distinct symptoms are listed for a reason. Preamble, key variation and truncation have three different causes, so the answer that fixes only one of them is not the answer.

Ask an engineering team which prompt version is running in production right now. In most organisations, **nobody can answer that.**

Amazon Bedrock Prompt Management

- ◆ Prompts are **stored resources with immutable versions**. Version 7 is version 7 forever.
- ◆ **The model binding lives inside the version**. Your code says "run prompt version 7"; the version decides which model that is.
- ◆ **Variables** make templates parameterised, so the structure is fixed and only the values change.
- ◆ Test and compare prompts in the **Prompt Builder** without deploying anything.
- ◆ **No additional charge** for storing prompts. You pay the model tokens when you run them.

ON THE EXAM

That second point is the powerful one. Because the model is a property of the stored version, changing model is a prompt deployment rather than a code deployment, which echoes AppConfig in module 1.

Four ways to consume a prompt version

PATH	HOW	NOTE
Direct invocation	Pass the prompt ARN as modelId to Converse	Model comes from the version
Retrieve then send	GetPrompt, then build the call yourself	When you need to inspect or modify it
Bedrock Flows	A Prompt node references a version	The multi-step path
Agents	Model left unspecified, supplied at orchestration	This path targets Agents Classic

ON THE EXAM

The Agents path points at Amazon Bedrock Agents Classic, which closed to new customers on 30 July 2026. Still examinable, and not where new work should go.

Who changed what, and who used it

- ◆ AWS CloudTrail records the management-plane actions. Prompt created, version published, prompt deleted, by which principal.
- ◆ **CloudWatch Logs** and **model invocation logging** record the runtime side: which prompt ran, with what input and output.
- ◆ **S3** as a template repository when prompts are managed as code alongside application source.
- ◆ **Approval workflows** gate publication, exactly as the Model Registry gated deployment in module 2.
- ◆ A prompt edited in the console with no version and no trail satisfies none of this.

ON THE EXAM

Learn the split: CloudTrail answers "who changed the prompt", model invocation logging answers "what did it produce". Compliance questions almost always need both.

A bank's advisory assistant is subject to regulatory review. The regulator has asked the bank to demonstrate, for any customer interaction in the past two years, which prompt text was in force at that moment, who approved it, and what the assistant actually said.

Prompts are currently hardcoded in Lambda functions and changed through ordinary code deployments. The bank's compliance team has proposed adding a comment above each prompt string recording who last changed it.

What should the bank implement?

- ☐ A Store prompts in Bedrock Prompt Management as versioned resources with an approval gate before publication, enable CloudTrail for management events, and enable model invocation logging for the runtime record
- ☐ B Keep prompts in code and rely on Git history plus code review records
- ☐ C Enable CloudTrail and model invocation logging, leaving prompts hardcoded
- ☐ D Store each prompt as a versioned object in Amazon S3 and read it at runtime

What should the bank implement?

- A

Prompt Management, approval gate, CloudTrail, invocation logging

Covers all three questions: the version records the text, the approval gate records who signed it off, and invocation logging records what was said.
- B

Git history and code review

Records the change in the repository. It cannot tie a specific customer interaction at a specific time to the prompt version that served it.
- C

Logging only, prompts hardcoded

Captures what was said, and CloudTrail records a Lambda deployment rather than a prompt change. Two of the three questions stay unanswered.
- D

Versioned S3 objects

Versions the text and provides no approval workflow, no model binding, and no link from an invocation to the object version used.

THE SIGNAL

Three questions were asked: what text, who approved, what was said. Score each option against all three. Only A answers every one, and B and C each answer exactly one.

Testing a prompt before it ships

- ◆ **A golden set.** Fixed inputs with known-good outputs. Without one, you are not testing, you are sampling.
- ◆ **Lambda verification.** Does the output parse, contain required fields, and satisfy assertions?
- ◆ **Step Functions for edge cases.** Empty input, hostile input, very long input, wrong language.
- ◆ **CloudWatch metrics for regression.** Track pass rate per prompt version and alarm when it falls.
- ◆ "I tried it a few times and it looked fine" is how prompt regressions reach production.

ON THE EXAM

Prompts are non-deterministic, so a single pass proves nothing. Run the golden set repeatedly and treat the pass rate as the metric, not the result of one run.

A governed prompt change

01

Draft

Edit in the Prompt Builder, compare against the current version.

02

Test

Run the golden set. Record pass rate, not a vibe.

03

Publish

Create an immutable version once the gate passes.

04

Approve

A named human signs off, and CloudTrail records it.

05

Monitor

Watch quality metrics after release, ready to pin back.

ON THE EXAM

Rollback is trivial here for the same reason it was in module 2: the previous version still exists. You repoint at version 6 rather than trying to remember what version 6 said.

A retailer changed a product-description prompt to improve tone. The change was tested by an engineer running four sample products through the Prompt Builder, all of which looked good. It shipped on a Friday.

Over the weekend, roughly one description in fifteen omitted the required allergen warning, which had previously appeared reliably. Nobody noticed until Monday. The team wants to prevent a recurrence without slowing routine prompt changes to a crawl.

What should the team put in place?

- ☐ A Require two engineers to review every prompt change before it ships
- ☐ B Build a golden set with assertions, run it automatically on every prompt version, gate publication on the pass rate, and alarm on the metric in production
- ☐ C Add "always include the allergen warning" to the prompt in bold capitals
- ☐ D Apply a Bedrock guardrail to block descriptions lacking an allergen warning

◆ PAUSE AND DECIDE

What should the team put in place?

- A

 Two-engineer review
Human review does not detect a one-in-fifteen probabilistic failure. Both reviewers would have looked at samples that passed, exactly as the first engineer did.
- B

 Golden set, automated gate, production alarm
A one-in-fifteen failure is a statistical property, so only a statistical test finds it. Automation also keeps routine changes fast.
- C

 Emphatic instruction
Might raise compliance from 93% to 97%, with no way to know. It adds no detection, so the next regression is discovered by customers again.
- D

 Guardrail on allergen warnings
Guardrails filter harmful content and denied topics. They do not assert that a required field is present in an otherwise benign description.

THE SIGNAL

"One in fifteen" is the whole question. Four samples had roughly a three-in-four chance of all looking fine, so the testing method could not have caught it however careful the engineer was.

Amazon Bedrock Flows

- ◆ **Renamed.** Prompt Flows became **Bedrock Flows** at GA, November 2024.
- ◆ A **visual builder** of nodes: prompts, knowledge bases, agents, Lambda, conditions.
- ◆ **Conditional branching** on model output. That is what makes it more than a chain.
- ◆ **Guardrails attach** to Prompt and Knowledge Base nodes in the builder.
- ◆ **Versioned and traceable.** InvokeFlow emits traces. Billed per node transition.

ON THE EXAM

The in-scope services list still says Prompt Flows. If one option says Prompt Flows and another says Bedrock Flows, they mean the same service.

Bedrock Flows or Step Functions?

BEDROCK FLOWS

Purpose-built for prompt chains

- Prompts, knowledge bases and guardrails as nodes
- Visual builder, low or no code
- Guardrails attach without wiring
- Bounded to Bedrock building blocks

STEP FUNCTIONS

General-purpose orchestration

- Any AWS service, any API, any error handler
- Retries, catch, parallel, map, wait
- Long-running and durable workflows
- More to build, and far fewer limits

THE TIEBREAKER

If the workflow is prompts, retrieval and branching, use Flows. The moment it needs approvals, external systems or sophisticated error handling, it is a Step Functions workflow.

Keeping a conversation coherent

- ◆ **Clarification workflows.** When the request is ambiguous, ask rather than guess. Step Functions orchestrates the loop.
- ◆ **Intent recognition.** Amazon Comprehend classifies the request cheaply before an expensive model sees it.
- ◆ **Conversation history in DynamoDB,** keyed by session with a TTL, as module 3 established.
- ◆ **Rewrite follow-ups into standalone queries** before retrieval, or the vector store sees a fragment.
- ◆ Summarise older turns rather than dropping them, so early detail survives a long conversation.

ON THE EXAM

Comprehend for intent is the cheap-tool answer again. Classifying into a known set of intents is a fixed, high-volume task, which is exactly where Comprehend beats a foundation model.

An insurer wants a claims triage workflow: classify an incoming claim, retrieve the matching policy wording, draft an assessment, and route high-value claims to a human adjuster who must approve or reject before anything is sent to the customer.

The approval step may take up to three business days. If the adjuster rejects, the workflow must return to the drafting step with their comments. The team wants error handling and retries on every external call.

Which orchestration should the insurer use?

- A Amazon Bedrock Flows, using a condition node to branch on claim value
- B AWS Step Functions, with a task token callback for the adjuster approval and retry policies on each state
- C A single Lambda function calling Bedrock repeatedly, with the approval handled by a polling loop
- D Bedrock Flows for the AI steps, invoked from a Lambda function that sleeps until approval arrives

◆ PAUSE AND DECIDE

Which orchestration should the insurer use?

A Bedrock Flows with a condition node

Handles the classify, retrieve and draft steps well. A three-day human pause with a rejection loop is outside what a prompt-chain builder is for.

C One Lambda with a polling loop

Lambda has a 15-minute maximum. A three-day wait is not something a function can hold, and all error handling becomes yours to write.

B Step Functions with a task token callback

The callback pattern is built for waiting on a human for days, and per-state retry policies give the error handling the team asked for.

D Flows called from a sleeping Lambda

Same 15-minute wall. Paying for a Lambda to sleep for three days would be the wrong answer even if it were possible.

THE SIGNAL

"Up to three business days" decides this on its own. Any answer involving a Lambda waiting is eliminated by the 15-minute limit before you consider anything else.

Caching and routing

- ◆ **Prompt caching** reuses a stable prefix across calls. AWS cites up to **90% cost** and **85% latency** reduction.
- ◆ Cached context stays available for about **five minutes after each access**, so it suits bursts, not sparse traffic.
- ◆ It needs a **stable prefix**. Put anything variable early in the prompt and the cache never hits.
- ◆ **Intelligent Prompt Routing** sends each request to one of two models in a family, based on predicted difficulty.
- ◆ AWS cites up to **30% cost** reduction. GA since April 2025, and you choose both models and the criteria.

ON THE EXAM

Both are prompt-shaped cost levers rather than model changes, which is why they live here rather than in module 14. Caching rewards good prompt structure directly.

A document review tool loads a 40-page contract into the context window, then answers a series of user questions about it. Users typically ask eight to twelve questions about the same contract within a few minutes.

Input token costs dominate the bill. The prompt is assembled as: current timestamp, user's name, the full contract text, then the question. An engineer has proposed reducing costs by summarising the contract before sending it, accepting some loss of detail.

What is the better change?

- | | |
|---|--|
| <p>☐ A Summarise the contract before sending it, as proposed</p> | <p>☐ B Reorder the prompt so the contract text comes first as a stable prefix, then enable prompt caching</p> |
| <p>☐ C Enable prompt caching without changing the prompt order</p> | <p>☐ D Switch to a smaller model with a larger context window</p> |

◆ PAUSE AND DECIDE

What is the better change?

- A

 Summarise the contract
Cuts tokens by destroying the detail the tool exists to answer questions about. It solves a cost problem by removing the product.
- B

 Reorder for a stable prefix, then cache
Eight to twelve questions in a few minutes over identical content is the exact caching profile, and reordering is what makes the prefix cacheable at all.
- C

 Cache without reordering
The timestamp changes on every call, so the prefix differs from the first character. The cache never hits and nothing improves.
- D

 Smaller model, larger context
Still reprocesses the whole contract on every question. It reduces the unit price of doing avoidable work.

THE SIGNAL

The prompt assembly order is stated deliberately. A timestamp at the front invalidates every cache entry, which is why C is the trap for anyone who knows caching exists but not how it works.

The whole domain on one slide

- ◆ **Selection** under five constraints: capability, latency, throughput, residency, cost.
- ◆ **Customisation** is a ladder. Knowledge problem, retrieve. Behaviour problem, fine-tune.
- ◆ **Data** must be validated upstream, because a model cannot signal bad input.
- ◆ **Retrieval** splits into build-time and query-time. Reach for the query-time knobs first.
- ◆ **Prompts** are production artefacts: versioned, tested against a golden set, approved and audited.

ON THE EXAM

That is 31% of the exam. Domain 2 starts next, and it assumes every piece of vocabulary from these six modules.